

Atividade Unidade - 03

Professor: Jefferson Gomes Dutra

Pontuação: 10 pts

Desenvolver uma **aplicação funcional**, utilizando os principais conceitos da programação orientada a objetos (POO). A aplicação deve conter um conjunto mínimo de funcionalidades que envolvam cadastro, consulta, alteração e remoção de dados (CRUD).

O **tema é livre**, ficando a seu critério escolher o contexto da aplicação (ex: gestão, simulação, controle, organização etc.).

Informações

- O foco principal da avaliação não é a quantidade de telas ou CRUDs, mas a qualidade da modelagem orientada a objetos e das regras de negócio
- O trabalho deve ser em grupo composto por 3-4 componentes, limitados a 10 grupos no total.
- Deve ser realizado de maneira colaborativa usando alguma plataforma git para controle de versão (Sugestão: Github).
- A ordem de apresentação será definida em sorteio e os grupos podem trocar a ordem desde que haja acordo.
 - As apresentações podem ser online, dependendo da disponibilidade da turma.
- Apenas uma pessoa do grupo vai enviar na tarefa do SIGAA, arquivo txt com o nome, matrícula e link do repositório, no REAME as instruções para rodar a aplicação, além do código fonte do projeto zipado.
- A linguagem é de escolha do grupo, contudo, se o grupo escolher uma linguagem diferente da usada na disciplina, talvez eu não possa ajudar com eventuais problemas.
- A apresentação do Sistema Funcionando é obrigatória, se não apresentar receberá nota 0.
- Serão avaliados o domínio dos conceitos de Programação Orientada a Objetos, a capacidade de explicar e justificar as decisões de modelagem e implementação adotadas no trabalho, a aplicação correta dos conteúdos vistos na disciplina, bem como a clareza da apresentação.

Planilha dos grupos:

<https://docs.google.com/spreadsheets/d/1gSKGrSUCFNp3aatlICa9fxJ97VSldnorVZE6BwgrVA4/edit?usp=sharing>

g

Requisitos Mínimos

1. Mínimo 9 Classes

- a. Devem ser usadas classes, não structs
- b. O sistema deve possuir ao menos 2 situações em que diferentes tipos compartilham parte do comportamento, mas possuem regras específicas (Polimorfismo).
- c. O sistema deve conter pelo menos 5 regras de negócio relevantes que envolvam validações, estados ou comportamentos dinâmicos (Regras de negócio).
- d. O sistema deve possuir operações que envolvam interação entre múltiplas classes e regras de negócio associadas.
- e. Tratamento de exceções
 - i. Exceções personalizadas para situações reais da aplicação
 - ii. Tratar validação de dados com exceções
- f. Diagrama de Classes – UML
 - i. Com todas as classes
 - ii. Com todas as relações
 - iii. Com as multiplicidades relevantes
- g. Salvar informações em arquivos
 - i. Registrar as informações salvas na aplicação em arquivos
 - ii. Quando a aplicação iniciar, carregar essas informações
- h. Pelo menos 1 classe deve ter estado dinâmico
 - i. Por exemplo: Pedido passa por diversos estados: ACEITO, PAGO, ENVIADO, CANCELADO e ENTREGUE.
 - ii. Deve ser implementada as regras para tratar a transição entre estados, por exemplo: Um pedido aceito não pode pular diretamente para entregue, ele tem que ser pago primeiro.

- i. Deve ser usada a sobrecarga dos operadores de >> (Leitura) e << (Escrita) para ler e exibir as informações das classes
 - j. Makefile para facilitar a compilação e execução do Código C++
2. Apresentação
- a. A apresentação do Sistema Funcionando é obrigatória, se não apresentar receberá 0.
 - b. O sistema deve suportar interação com o usuário, onde o usuário deve interagir com o sistema inserindo as informações e opções via terminal.

Observações

1. Todas as heranças, implementações e polimorfismo devem ser de classes próprias. Herança e implementação de classes oriundas de bibliotecas, frameworks e afins não serão consideradas.
2. Heranças com classes filhas sem membros específicos não são aceitos.

```
class Pessoa {  
public:  
    string nome;  
    string cpf;  
};  
  
class Aluno : public Pessoa {  
  
};  
  
class Professor : public Pessoa {  
  
};
```

Não aceito. Classes filhas não tem sentido em existir se não tem atributos ou comportamentos próprios

```

class Pessoa {
public:
    string nome;
    string cpf;
};

class Aluno : public Pessoa {
    string matricula;
};

class Professor : public Pessoa {
    string dataFinalContrato;
};

```

Aceito. Classes filhas têm atributos ou comportamentos próprios.

3. Os comportamentos polimórficos devem estar relacionados a comportamentos relevantes do contexto do sistema. Como por exemplo regras de negócio.

```

class Exibicao {
public:
    virtual void exibir() = 0;
};

class Aluno : public Exibicao {
    string matricula;

    void exibir() {
        cout << matricula << endl;
    }
};

class Professor : public Exibicao {
    string dataFinalContrato;

    void exibir() {
        cout << dataFinalContrato << endl;
    }
};

```

Não aceito.

```

class CalculadoraDesconto {
public:
    virtual double calcular(double valor) = 0;
};

class DescontoAluno : public CalculadoraDesconto {

    double calcular(double valor) {
        return valor * 0.9;
    }
};

class DescontoProfessor : public CalculadoraDesconto {

    double calcular(double valor) {
        return valor * 0.8;
    }
};

```

4. Enums e Classes apenas com valores constantes não serão contabilizados como parte das 9 classes necessárias

```

enum class StatusPedido {
    Pendente,
    Pago,
    EmSeparacao,
    Enviado,
    Entregue,
    Cancelado
};

```

Não aceito.

```

class StatusPedido {
public:
    static const int Pendente = 0;
    static const int Pago = 1;
    static const int Enviado = 2;
    static const int Cancelado = 3;
};

```

Não aceito.

Cada item receberá nota proporcional *se não for completamente implementado* e 0 se não for implementado.

Apresentação

- Diagrama
- Código
- Sistema Funcionando